

TD-gGA 論文再現パッケージ 設計図

コード構成・データフロー・各ファイルの責務

td_gGA プロジェクト

2026 年 7 月 8 日 (2026 年 7 月 13 日: 配布パッケージ向けにファイル名を更新)

1 全体像

注記 (2026-07-13): 本文書は開発時のワークスペース (`td_gGA_solver_paperconv.py` 等、Route A/C の名残があるファイル名) から、第三者に配布するための本パッケージ (`td_gGA_solver.py` 等、役割がわかる名前) 向けに更新したもの。ファイル名以外の物理・アルゴリズムの内容に変更はない。旧名との対応は同梱の `README.md` を参照。

本ワークスペースは Guerci–Capone–Lanata, PRR **5**, L032023 (2023) の TD-gGA (時間依存ゴースト Gutzwiller 近似) の検証済み実装である。設計は次の 3 層に分離されている:

層	役割	成果物の置き場
静的計算	平衡 gGA 解 (初期条件の材料)	<code>static_cache/B{B}_Ui{...}_T{...}.npz</code>
時間発展	クエンチ後の TDVP 積分	<code>data/B{B}/quench_Ui{...}_Uf{...}.npz</code>
プロット	保存データの可視化のみ	<code>*.png</code>

各層は独立に再実行できる。静的解はキャッシュされ ($B = 5$ で約 25 分、 $B = 7$ で数時間の計算が 1 回きりになる)、プロットは再計算なしに何度でも描き直せる。

2 物理の要点 (実装が解いている方程式)

コードの規約 ($n(\omega)$ は標準 Fermi 行列、 $\Delta_e = \langle f^\dagger f \rangle$ 電子規約) で、 $\Lambda = 0$ ゲージの複素 TDVP:

$$\Delta_e = \langle f^\dagger f \rangle_\Phi, \quad g = [\Delta_e(1 - \Delta_e)]^{1/2}, \quad (1)$$

$$R = g^{-1} \langle f^\dagger c \rangle_\Phi, \quad B = \sum_f w_f \omega_f n_f \quad (\text{複素のまま}), \quad (2)$$

$$D = \overline{g^{-1} B R}, \quad M = R D^\top + \overline{D} R^\dagger, \quad (3)$$

$$\Lambda^c = +(\text{dg} \Delta_e[M])^\top \quad (\text{Loewner 閉形式}), \quad (4)$$

$$i \partial_t \Phi = \hat{H}_{\text{emb}}[D, \Lambda^c] \Phi, \quad \partial_t n(\omega) = -i\omega [R R^\dagger, n(\omega)]. \quad (5)$$

この組は拘束 $F_2 = \|\Delta_e - \sum_f w_f n_f\|$ とエネルギーを厳密に保存する (証明は `derivation_F2_conservation_proof`. §3, §7. 鍵は Loewner 恒等式 $[\Delta, \text{dg}[M]] = [g(\Delta), M]$)。

$\Lambda = 0$ ゲージ = 自由度の厳密消去 (2026-07-09 追記)

Λ はエルミート $B \times B$ 行列で、一般には B^2 個の実自由度を持つ。変分表示のうえでは Λ は物理的な観測量を変えないゲージ冗長度であり、上記 Loewner 恒等式のおかげで $\Lambda \equiv 0$ というゲージを初期時刻だけでなく全時刻にわたって厳密に維持できる (近似ではない)。この恒等式は Δ_e ($= U_i$ や状態そのもの) に依らない一般的な代数関係なので、「 U_i が小さいから許された」という話ではない (実測でも $B = 3$, $U_i = 0.05 \rightarrow U_f = 2.55$ の大クエンチで $|\Delta E/E|_{\max} = 2.3 \times 10^{-7}$)。

結果として力学変数は $(\Phi, n(\omega))$ だけになり、 Λ の B^2 自由度は毎ステップの root-find (旧 Route C 方式) なしに完全に消去される。旧 Route C が $B \geq 3$ でエネルギーを大きく破っていたのは、この消去が発見される前に Λ を独立変数として反復的に解いていたことに加え、`solve_lambda_differential` 内の符号バグ (`derivation_F2_conservation_proof.md` §6) が重なったためであり、「動的 Λ 方式が原理的に不可能」だったわけではない。ただ、上記の厳密消去が使える以上、それを使わない実装 (Route C) を維持する理由がない。

3 ソルバー核：td_gGA_solver.py

唯一の「正しい」ソルバー。主要関数と対応する式：

関数	責務
<code>solve_static(B, U_i, T)</code>	静的 gGA 平衡解 (<code>GA.optimize_selfc</code> を呼ぶ薄いラップ)
<code>save_static / load_static</code>	静的解の必要成分 ($\Phi_0, \Lambda, D, \Lambda^c, R, \text{fdaggerc}, \text{ffdagger}$) を npz へ保存／GA を最適化なしで再構成して注入
<code>get_static(B, U_i, T)</code>	キャッシュがあれば load、なければ solve して save
<code>hole_cplx(Phi, params)</code>	$\langle f_j f_i^\dagger \rangle$ (スピン平均、複素のまま。dense/sparse 自動判別)
<code>fdc_cplx(Phi, params)</code>	$\langle f^\dagger c \rangle$ (同上)
<code>g_inv_of(Delta)</code>	$[\Delta(1 - \Delta)]^{-1/2}$ (複素エルミート安全な eigh 実装)
<code>loewner_dg(Delta, M)</code>	Fréchet 微分 $\text{dg}[M]$ の Loewner 閉形式 (Λ^c 用。基底ループ・least_squares 不要)
<code>compute_RDLc_paper</code>	式 (1)–(4) : $\Phi, n \mapsto R, D, \Lambda^c, B$
<code>build_H_emb_paper</code>	$\hat{H}_{\text{emb}}$ を密行列で構築 ($B \leq 5$)
<code>apply_H_emb_sp</code>	$\hat{H}_{\text{emb}} \Phi$ を疎行列積の和で直接評価 ($B \geq 7$ 、近似なし)
<code>prepare_sparse_params</code>	演算子を疎のまま保持する軽量 params ($B \geq 6$ で自動選択)
<code>rhs_paper(t, y, params)</code>	TDVP の右辺：式 (5) (dense/sparse をディスパッチ)
<code>run(B, U_i, U_f, ...)</code>	1 本のクエンチの全工程：初期条件構築 $\rightarrow$ RK4 積分 $\rightarrow$ 観測量記録。戻り値 <code>rec</code> に $t, d, E, \Delta E/E , F_2, \sqrt{D^\dagger D}$, ghost split, eig Λ^c
<code>save_run(rec, ...)</code>	<code>rec</code> を <code>data/B{B}/</code> へ保存

run() 内の初期条件構築（重要ポイント）

1. Φ_0 = 静的 ED の基底状態 (npz 演算子基底)。 $\Delta_{e,0}, R_0$ を Φ_0 から再計算。
2. ゲージ回転 $W = \text{permmat} \cdot \text{phasemat} \cdot \text{transmat}^T$ (`fix_gauge` の変換行列) で $\Lambda_{\text{eq}} = W \Lambda_{\text{imp}} W^T$ 。
検証として $\|W R_{\text{imp}} - R_0\|$ を表示。
3. $\pm \Lambda_{\text{rot}}$ 符号自動選択：半充填の PH 対称性により (Λ, n) と $(-\Lambda, \text{PH 鏡映 } n)$ が縮退した同値レベルなので、両候補で F_2^{init} を測り小さい方を採用 ($> 10^{-3}$ なら root-refine)。 $B = 3$ は +、 $B = 5$ は - が正解だった。
4. $n_0(\omega) = f(\omega R_0 R_0^T + \Lambda_{\text{eq}})$ 、`ic_mod` フックで人工変形も可能（縮退多様体探索用）。

4 サポートモジュール（Lanata 系レガシー。変更禁止）

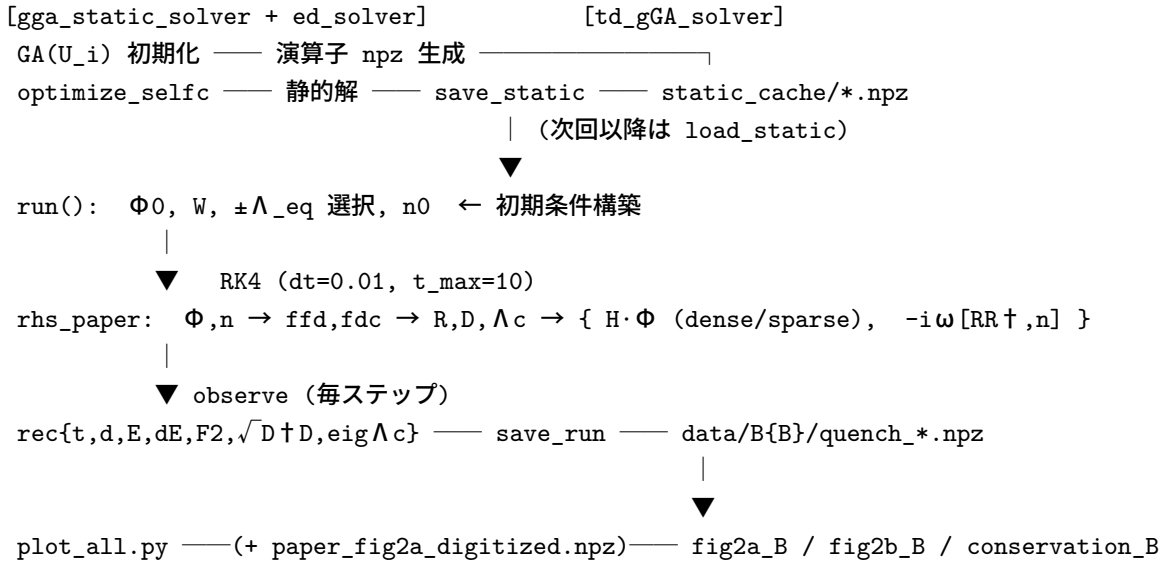
ファイル	役割
<code>gga_static_solver.py</code>	静的 gGA ソルバー本体（クラス GA）。 <code>optimize_selfc</code> (R, Λ の <code>least_squares</code>)、 <code>calc_Delta</code> ($\int \rho f(\omega R R^T + \Lambda)$)、 <code>calc_D</code> 、 <code>calc_Lmbdac</code> (実対称 Loewner)、 <code>fix_gauge</code> (Λ^c 対角化・ D 符号・並べ替えのゲージ固定)
<code>ed_solver.py</code>	埋め込みハミルトニアン of 厳密対角化。 <code>build_creation_ops</code> (Fock 空間の $f^\dagger$)、 <code>build_op_prods</code> (一体演算子積を npz 保存。 B ごとに同名上書き $\Rightarrow$ 異なる B の並走禁止)、 <code>build_Hemb</code> (疎行列で $\hat{H}_{\text{emb}}$ 組立、 $+10\hat{S}^2$ ペナルティ付き)、 <code>solve_Hemb</code> (<code>primme.eigsh</code> による疎対角化 — $B = 7$ でもそのまま動く)、 <code>calc_density_matrix</code>
<code>convenience_routines.py</code>	<code>calc_C</code> (Fermi 行列関数)、 <code>funcMat</code> 、 <code>dF</code> (Loewner)、実対称基底の生成など
<code>lattice.py</code>	半円 DOS 等の格子情報
<code>tdvp_helpers.py</code> (旧	<code>setup_frequency_grid</code> (Gauss-Legendre $\times$ 半円 DOS)、
<code>td_gGA_solver_routeA.py</code>	<code>pack_state/unpack_state</code> 、 <code>prepare_full_params</code>
+	(dense 演算子ロード、 $B \leq 5$)、 <code>compute_fdaggerc_sp</code> 、
<code>td_gGA_solver_routeC.py</code>)	<code>compute_ffdagger_sp</code> ($\langle f^\dagger f \rangle$)、 <code>solve_lambda_from_F2</code> (Λ root-finding、初期条件 Λ_{eq} の算出用)

注意：`tdvp_helpers.py` は元々 `tdvp_core.py` / `tdvp_sparse.py` という 2 ファイルで、それぞれ独立した TD ソルバー本体（開発時の呼称で Route A / Route C）だったが、規約バグ（転置・共役・符号の混在）で $B \geq 3$ の保存則・ B 依存性の再現に失敗していた。2026-07-13 の配布パッケージ整理でこの未使用のドライバ本体は削除し、`td_gGA_solver.py` が実際に `import` して使うヘルパー関数のみを残した（2 ファイル合計 1453 行 $\rightarrow$ 260 行）。統合後もファイル間の依存がゼロ（互いに `import` し合っていないかった）ことを確認したうえで、分割する意味がなくなったため 1 ファイルに統合した（260 行）。

5 実行スクリプトとプロット

ファイル	役割
run_quench.py	CLI: <code>--B 7 --dU 1.25 1.5 2.0 2.5 [--bare] [--tmax] [--dt]</code> 。get_static (キャッシュ) → run → save_run
plot_all.py	data/ を全部拾って fig2a_B.png ($d(t)$ 、論文デジタイズ点重ね)・fig2b_B.png ($\text{eig } \Lambda^c, \sqrt{D^\dagger D}$)・conservation_B.png を生成。計算ゼロ
run_B135.py	一括実行版 ($B = 1, 3, 5$ 静的 +TD+ プロット。旧形式)
run_fig2a.py, run_fig2b.py	単発の図用ランナー (plot_all 以前の形式)
compare_digitized.py	論文 Fig.2a を PDF からデジタイズし (軸は目盛りマークで校正)、我々の曲線と定量比較
run_convergence.py	$N_{\text{freq}}/\text{dt}/T$ 感度チェック
run_dU_definition.py	$U_f = U_i + \delta U$ か $U_f = \delta U$ かの判別 ($B = 1$ 周期)
saddle_scan.py	静的鞍点のシード探索 (結果: 引き込み点は単一)

6 データフロー



7 データ形式

static_cache/B{B}_Ui{U_i}_T{T}.npz : eig_vec(Φ_0), fdaggerc, ffdagger, Lmbda, D, Lmbdac, R, メタ (B, U_i, T_gGA)。

data/B{B}/quench_Ui{..}_Uf{..}.npz : t, d, E, dE($|\Delta E/E|$), F2, sqDtD, Dsplit, eigLc (形 $N_t \times B$) , メタ (B, U_i, U_f)。

`paper_fig2a_digitized.npz` : 論文 Fig.2a のデジタイズ (tg/dg_{\delta U}=B1 破線, tr/dr_{\delta U}=B3 赤)。`paper_fig2a_B57_digitized.npz` : 同 blue(B=5)/green(B=7)、 $\delta U = 2.0, 2.5$ のみ。

8 数値セッティング一覧

モデル・物理パラメータ

項目	既定値	備考
模型	単一バンド Hubbard, 半充填	$\hat{H} = \frac{U}{2} \sum_i (\hat{n}_i - 1)^2 - J \sum c^\dagger c$
格子	ベレー格子 (配位数 ∞)	半円 DOS $\rho(\omega) = \frac{2}{\pi} \sqrt{1 - \omega^2}$
エネルギー単位	半バンド幅 $D = 1$	時間単位は D^{-1} (論文と同一)
U_i	0.05	論文脚注 [55] ($U=0$ は $B \geq 3$ で縮退・不安定)
U_f	$U_i + \delta U$ (--bare で δU)	論文の δU ラベルの定義は図の精度で判別不能 ($B=1$ 周期の判別テスト参照)
温度 T	0.003	Fermi 関数の平滑化 (準 $T=0$)。感度 $\Delta d \sim 1.5 \times 10^{-5}$

時間発展 (TD)

項目	既定値	備考
積分器	固定刻み RK4	各 RHS 評価で Φ を規格化
dt	0.01	感度チェック済み ($dt=0.005$ で $\Delta d \sim 3 \times 10^{-6}$)。 $B=5$ の F_2 ドリフト ($\sim 10^{-3}$) を下げたい場合は半分に
$t_{\max}$	10	論文 Fig.2 と同範囲
N_{freq}	60	Gauss-Legendre 点 $\times \rho(\omega)$ 重み。収束確認済み (150/300 で $\Delta d \leq 1.6 \times 10^{-4}$)
状態変数	$\Phi \in \mathbb{C}^{\dim \Phi}, n(\omega) \in \mathbb{C}^{N_{\text{freq}} \times B \times B}$	複素のまま伝播 (np.real 禁止が鉄則)

静的計算・ED・初期条件

項目	既定値	備考
静的最適化	<code>least_squares</code> (R : B 個 + Λ : $B(B+1)/2$ 個)	シードは $R = [1, 0, \dots]$, Λ 係数 = $[1, 0, \dots]$ 。収束残差 $\text{Max.error} \sim 5 \times 10^{-4}$ ($B=3$) が初期条件の質の下限
ω 積分 (静的)	<code>scipy.integrate.quad</code> , <code>epsabs</code> 10^{-14}	<code>calc_Delta</code> 内
ED	<code>primme.eigsh</code> , <code>num_eig</code> =2, <code>tol</code> 10^{-12} , 'SA'	疎行列のまま。+10 $\hat{S}^2$ ペナルティで一重項基底状態を選択
EH セクター	$1+B$ フェルミオン (半充填)	$\dim \Phi = \binom{2(1+B)}{1+B}$
Λ_{eq} 構築	ゲージ回転 $W\Lambda_{\text{imp}}W^T$ + $\pm$ 符号自動選択	$F_2^{\text{init}} > 10^{-3}$ なら <code>solve_lambda_from_F2</code> (<code>tol</code> 10^{-10}) で <code>refine</code>
行列関数の保護	Δ 固有値を $[10^{-12}, 1-10^{-12}]$ にクリップ	Loewner 核の縮退判定は $ \Delta\lambda > 10^{-9}$ (それ以下は g')

到達している保存精度 (既定セッティングでの実測)

ケース ($U_i=0.05 \rightarrow U_f=2.55$, $t=10$)	$ \Delta E/E _{\text{max}}$	F_2^{max}
$B = 1$	8×10^{-10}	1.2×10^{-5} (一定)
$B = 3$	2.3×10^{-7}	2.6×10^{-5} (一定)
$B = 5$	2.8×10^{-5}	1.2×10^{-3} (RK4 離散化誤差、 dt で低減可)

9 計算コストの目安

B	$\dim \Phi$	演算子生成	静的 (初回のみ)	TD 1 クエンチ	パス
1	6	一瞬	数秒	数秒	dense
3	70	数秒	~1.5 分	数秒	dense
5	924	~1 分	~25 分	~20 分	dense (~1GB)
7	12870	~10 分	数時間 (見込み)	数分 (見込み)	sparse

疎行列パスは近似なし (一体演算子の Fock 行列は 1 列あたり非零 ≤ 1)。 $B = 3$ で dense と厳密一致 ($\max |\Delta d| = 2 \times 10^{-13}$) を検証済み。

10 運用ルール

- 異なる B の計算を同一ディレクトリで並走させない (演算子 npz が同名上書き)。
- 論文との比較は PPT 重ねでなく `compare_digitized.py` の枠組みで定量的に。
- 新しい観測量は `run()` の `observe()` に追加 $\rightarrow$ `rec` に記録 $\rightarrow$ `plot_all.py` に描画を足す、が標準手順。
- 理論・経緯はそれぞれ `derivation_F2_conservation_proof.md`・`POSTMORTEM_2026-07-07.md` を参照 (開発時の全経緯を追った `STATUS.md`・`ROADMAP.md` は本配布パッケージには含めていな

い。必要な場合は開発ワークスペース側を参照)。